MON APPRENTISSAGE — SERIES PYTHON

Python

Du zéro à l'expert

Chapitre 30 — Programmation Fonctionnelle

functools, operator, immutabilité — penser en transformations

Auteur :

KETOTSA AMÉVI CLAUDE

Date de publication :

28 mars 2026

Publié dans le cadre de mon parcours d'apprentissage Python

Table des matières

Chapitre 30 — Programmation Fonctionnelle **2**
0.1 Fonctions pures et immutabilité 2
0.2 functools — La boîte à outils fonctionnelle 3
0.3 operator — Fonctions pour les opérateurs 6
0.4 Composition de fonctions 7
0.5 Currying et application partielle avancée 9
0.6 Monades légères — Maybe et Result 10
0.7 Ce qui m'a le plus surpris dans ce chapitre 12
0.8 Résumé du Chapitre 30 12

Chapitre 30 — Programmation Fonctionnelle

À retenir

La **programmation fonctionnelle** est un paradigme qui traite le calcul comme l'évaluation de **fonctions mathématiques pures** — sans état mutable, sans effets de bord. Python n'est pas un langage fonctionnel pur, mais il emprunte généreusement à ce paradigme : fonctions de première classe, lambdas, `map`, `filter`, `functools`. Maîtriser ces outils permet d'écrire du code plus **concis, prévisible et testable**.

Fonctions pures et immutabilité

Fonctions pures — sans effets de bord

```python
# Fonction IMPURE     modifie l'etat exterieur
total_global = 0

def ajouter_impure(valeur: float) -> None:
    global total_global
    total_global += valeur    # effet de bord !

# Fonction PURE     meme entree     meme sortie, toujours
def ajouter_pure(total: float, valeur: float) -> float:
    return total + valeur    # pas d'effet de bord

# Avantages des fonctions pures :
# 1. Testables en isolation
# 2. Composables
# 3. M morisables (cache)
# 4. Parall lisables

# Immutabilite     ne jamais modifier les entrees
# MAUVAIS     modifie la liste en place
def trier_notes_mal(notes: list[float]) -> list[float]:
    notes.sort()     # modifie l'original !
    return notes

# BON     retourne une nouvelle liste
def trier_notes_bien(notes: list[float]) -> list[float]:
    return sorted(notes)    # copie + tri

notes = [15.0, 18.0, 12.0, 9.0]
triees = trier_notes_bien(notes)
print(notes)     # [15.0, 18.0, 12.0, 9.0]  inchange !
print(triees)    # [9.0, 12.0, 15.0, 18.0]
```

```python

# Transformer sans muter      pipeline de donnees
etudiants = [
    {"prenom": "Alice", "note": 15.0},
    {"prenom": "Bob",   "note": 18.0},
    {"prenom": "Eve",   "note": 9.0},
    {"prenom": "Charlie","note": 12.0},
]

# Approche fonctionnelle      chaque etape retourne une
# nouvelle collection, l'originale est preservee
admis      = [e for e in etudiants if e["note"] >= 10]
tries      = sorted(admis, key=lambda e: e["note"],
                    reverse=True)
prenoms    = [e["prenom"] for e in tries]
print(prenoms)  # ['Bob', 'Alice', 'Charlie']
print(len(etudiants))   # 4      toujours intact
```

functools — La boîte à outils fonctionnelle

functools.partial — application partielle

```python
from functools import partial
import math

# partial()      fixer certains arguments d'une fonction
def puissance(base: float, exposant: float) -> float:
    return base ** exposant

# Cr er des fonctions specialisees
carre    = partial(puissance, exposant=2)
cube     = partial(puissance, exposant=3)
racine   = partial(puissance, exposant=0.5)

print(carre(5))      # 25.0
print(cube(3))       # 27.0
print(racine(16))    # 4.0

# Tres utile avec map(), sorted(), filter()
nombres = [1, 4, 9, 16, 25]
print(list(map(racine, nombres)))
# [1.0, 2.0, 3.0, 4.0, 5.0]

# partial avec des fonctions a plusieurs arguments
def formater(valeur: float, decimales: int,
             symbole: str) -> str:
    return f"{valeur:.{decimales}f}{symbole}"

```

```python
formater_euro    = partial(formater, decimales=2,
                           symbole="    ")
formater_pourcent = partial(formater, decimales=1,
                            symbole=" %")

prix = [1234.5, 99.99, 0.5]
print(list(map(formater_euro, prix)))
# ['1234.50    ', '99.99    ', '0.50    ']

taux = [12.345, 5.6, 100.0]
print(list(map(formater_pourcent, taux)))
# ['12.3 %', '5.6 %', '100.0 %']
```

functools.lru_cache – memoisation

```python
from functools import lru_cache, cache
import time

# Sans cache     recalcul a chaque appel
def fibonacci_lent(n: int) -> int:
    if n <= 1:
        return n
    return fibonacci_lent(n-1) + fibonacci_lent(n-2)

# Avec lru_cache     memoisation automatique
@lru_cache(maxsize=128)
def fibonacci(n: int) -> int:
    if n <= 1:
        return n
    return fibonacci(n-1) + fibonacci(n-2)

# Python 3.9+     cache illimite
@cache
def factorielle(n: int) -> int:
    if n == 0:
        return 1
    return n * factorielle(n-1)

# Mesure de performance
t0 = time.perf_counter()
_ = fibonacci_lent(35)
print(f"Sans cache : {time.perf_counter()-t0:.3f}s")

t0 = time.perf_counter()
_ = fibonacci(35)
print(f"Avec cache : {time.perf_counter()-t0:.6f}s")
#     Plusieurs ordres de grandeur plus rapide !

# Inspecter le cache
```

```python
print(fibonacci.cache_info())
# CacheInfo(hits=33, misses=36, maxsize=128, currsize=36)

# Cache pour des requetes couteuses
@lru_cache(maxsize=64)
def charger_config(chemin: str) -> dict:
    """Lu une seule fois     resultat mis en cache."""
    import json
    with open(chemin) as f:
        return json.load(f)
```

functools.reduce et wraps

```python
from functools import reduce, wraps
from typing import Callable, TypeVar

T = TypeVar("T")

# reduce()     plier une sequence en une valeur
nombres = [1, 2, 3, 4, 5]

produit = reduce(lambda acc, x: acc * x, nombres)
print(f"Produit : {produit}")   # 120

maximum = reduce(lambda a, b: a if a > b else b, nombres)
print(f"Maximum : {maximum}")   # 5

# Concatener une liste de listes
listes = [[1,2], [3,4], [5,6], [7,8]]
aplatie = reduce(lambda acc, l: acc + l, listes)
print(f"Aplatie : {aplatie}")   # [1,2,3,4,5,6,7,8]

# wraps     preserver les metadonnees des decorateurs
def logger(fn: Callable) -> Callable:
    @wraps(fn)   # INDISPENSABLE !
    def wrapper(*args, **kwargs):
        print(f"    Appel : {fn.__name__}")
        resultat = fn(*args, **kwargs)
        print(f"    Retour : {resultat!r}")
        return resultat
    return wrapper

@logger
def additionner(a: float, b: float) -> float:
    """Additionne deux nombres."""
    return a + b

additionner(3, 4)
print(additionner.__name__)  # additionner (pas wrapper)
```

```python
print(additionner.__doc__)    # Additionne deux nombres.
```

operator — Fonctions pour les opérateurs

operator — remplacer les lambdas simples

```python
from operator import (itemgetter, attrgetter,
                      methodcaller, add, mul,
                      lt, gt, eq, neg)
from functools import reduce
from dataclasses import dataclass

@dataclass
class Etudiant:
    prenom: str
    note:   float
    ville:  str

etudiants = [
    Etudiant("Alice",   15.5, "Abidjan"),
    Etudiant("Bob",     18.0, "Lom  "),
    Etudiant("Charlie", 12.0, "Dakar"),
    Etudiant("Diana",   9.5,  "Accra"),
]

# attrgetter      acceder a un attribut
get_note   = attrgetter("note")
get_prenom = attrgetter("prenom")

# Trier par note
par_note = sorted(etudiants, key=get_note, reverse=True)
for e in par_note:
    print(f"  {e.prenom:10} {e.note}")

# Trier par ville puis par note
par_ville_note = sorted(
    etudiants,
    key=attrgetter("ville", "note")
)

# itemgetter      acceder a une cle de dict ou index
donnees = [
    {"prenom": "Alice", "note": 15.5, "rang": 2},
    {"prenom": "Bob",   "note": 18.0, "rang": 1},
    {"prenom": "Eve",   "note": 12.0, "rang": 3},
]

get_note_dict = itemgetter("note")
```

```python
par_note_dict = sorted(donnees, key=get_note_dict,
                       reverse=True)
for d in par_note_dict:
    print(f"  {d['prenom']:10} {d['note']}")

# Extraire plusieurs champs
get_rang_note = itemgetter("rang", "note")
print(get_rang_note(donnees[0]))   # (2, 15.5)

# methodcaller     appeler une methode
mots = ["python", "est", "g nial", "et", "puissant"]
majuscules = list(map(methodcaller("upper"), mots))
print(majuscules)

capitalises = list(map(methodcaller("capitalize"), mots))
print(capitalises)

# Operateurs arithmetiques comme fonctions
print(add(3, 4))     # 7
print(mul(3, 4))     # 12
print(neg(5))        # -5

# Reduce avec operator
from operator import add, mul
somme   = reduce(add, [1, 2, 3, 4, 5])   # 15
produit = reduce(mul, [1, 2, 3, 4, 5])   # 120
print(somme, produit)
```

Composition de fonctions

Composer des fonctions — pipelines de données

```python
from typing import Callable, TypeVar
from functools import reduce

T = TypeVar("T")

def composer(*fonctions: Callable) -> Callable:
    """Compose N fonctions :   f  gh  (x) = f(g(h(x)))"""
    def appliquer(x):
        return reduce(lambda v, f: f(v),
                      reversed(fonctions), x)
    return appliquer

def pipeline(*fonctions: Callable) -> Callable:
    """Pipeline gauche-droite : x     f     g     h"""
    def appliquer(x):
        return reduce(lambda v, f: f(v), fonctions, x)
```

```python
    return appliquer

# Fonctions pures simples
def nettoyer(texte: str)     -> str: return texte.strip()
def minuscules(texte: str)   -> str: return texte.lower()
def tokeniser(texte: str)    -> list: return texte.split()
def filtrer_courts(mots: list) -> list:
    return [m for m in mots if len(m) > 3]
def compter(mots: list)      -> dict:
    freq = {}
    for m in mots:
        freq[m] = freq.get(m, 0) + 1
    return freq

# Composer le pipeline de traitement texte
traiter_texte = pipeline(
    nettoyer,
    minuscules,
    tokeniser,
    filtrer_courts,
    compter
)

texte = "  Python est un langage Python tres puissant  "
resultat = traiter_texte(texte)
print(resultat)
# {'python': 2, 'langage': 1, 'tres': 1, 'puissant': 1}

# Pipeline de traitement numerique
import math

traiter_notes = pipeline(
    lambda notes: [n for n in notes if 0 <= n <= 20],
    sorted,
    lambda notes: notes[1:-1] if len(notes) > 4
                  else notes,   # supprimer extremes
    lambda notes: sum(notes) / len(notes) if notes else 0,
    lambda moy: round(moy, 2)
)

notes_brutes = [15, -1, 18, 200, 12, 9, 17, 14]
print(f"Moyenne nettoy e : {traiter_notes(notes_brutes)}")
```

Currying et application partielle avancée

Currying — transformer les arguments

```python
from typing import Callable, TypeVar

# Currying manuel      une fonction     N args
# devient N fonctions     1 arg
def curry_ajouter(a: float) -> Callable:
    def ajouter_a(b: float) -> float:
        return a + b
    return ajouter_a

ajouter_10 = curry_ajouter(10)
ajouter_5  = curry_ajouter(5)

print(ajouter_10(3))    # 13
print(ajouter_5(7))     # 12
print(list(map(ajouter_10, [1, 2, 3, 4, 5])))
# [11, 12, 13, 14, 15]

# Decorateur curry generique
def curry(fn: Callable) -> Callable:
    """Transforme f(a,b,c) en f(a)(b)(c)."""
    import inspect
    arity = len(inspect.signature(fn).parameters)

    def accumuler(*args):
        if len(args) >= arity:
            return fn(*args[:arity])
        return lambda *plus: accumuler(*(args + plus))

    return accumuler

@curry
def multiplier(a: float, b: float, c: float) -> float:
    return a * b * c

print(multiplier(2)(3)(4))     # 24
print(multiplier(2, 3)(4))     # 24
print(multiplier(2)(3, 4))     # 24
print(multiplier(2, 3, 4))     # 24

doubler_fois_trois = multiplier(2)(3)
print(doubler_fois_trois(5))    # 30
print(doubler_fois_trois(10))   # 60

# Application      transformer des donnees
@curry
def filtrer_par(cle: str, valeur, collection: list) -> list:
```

```python
    return [item for item in collection
            if item.get(cle) == valeur]

etudiants = [
    {"prenom": "Alice", "ville": "Abidjan", "note": 15},
    {"prenom": "Bob",   "ville": "Lom ",    "note": 18},
    {"prenom": "Eve",   "ville": "Abidjan", "note": 12},
]

abidjanais   = filtrer_par("ville")("Abidjan")
lomeens      = filtrer_par("ville")("Lom ")

print(abidjanais(etudiants))
print(lomeens(etudiants))
```

Monades légères — Maybe et Result

Maybe et Result — gérer les erreurs fonctionnellement

```python
from typing import TypeVar, Generic, Callable, Optional
from dataclasses import dataclass

T = TypeVar("T")
U = TypeVar("U")

# Maybe     encapsuler une valeur potentiellement absente
@dataclass(frozen=True)
class Maybe(Generic[T]):
    _valeur: Optional[T]

    @classmethod
    def de(cls, valeur: Optional[T]) -> "Maybe[T]":
        return cls(valeur)

    @classmethod
    def vide(cls) -> "Maybe[T]":
        return cls(None)

    def bind(self, fn: Callable[[T], "Maybe[U]"]
             ) -> "Maybe[U]":
        if self._valeur is None:
            return Maybe.vide()
        return fn(self._valeur)

    def map(self, fn: Callable[[T], U]) -> "Maybe[U]":
        if self._valeur is None:
            return Maybe.vide()
        return Maybe.de(fn(self._valeur))
```

```python

    def ou(self, defaut: T) -> T:
        return self._valeur if self._valeur is not None \
               else defaut

    def __repr__(self) -> str:
        if self._valeur is None:
            return "Maybe(vide)"
        return f"Maybe({self._valeur!r})"


# Utilisation     chainer sans v rifier None a chaque etape
base = {
    "alice": {"prenom": "Alice", "note": 15.5},
    "bob":   {"prenom": "Bob",   "note": 18.0},
}

def trouver(email: str) -> Maybe[dict]:
    return Maybe.de(base.get(email))

def extraire_note(profil: dict) -> Maybe[float]:
    return Maybe.de(profil.get("note"))

def calculer_mention(note: float) -> Maybe[str]:
    if note >= 16: return Maybe.de("Tr s Bien")
    if note >= 10: return Maybe.de("Passable+")
    return Maybe.de(" chec ")

# Pipeline sans aucune verification None explicite
mention_alice = (trouver("alice")
                 .bind(extraire_note)
                 .bind(calculer_mention)
                 .ou("Inconnu"))

mention_inconnu = (trouver("inconnu@ci")
                   .bind(extraire_note)
                   .bind(calculer_mention)
                   .ou("Etudiant introuvable"))

print(f"Alice   : {mention_alice}")
print(f"Inconnu : {mention_inconnu}")
```

Ce qui m'a le plus surpris dans ce chapitre

Ce qui m'a résisté

Trois révélations de ce chapitre :

1. Les fonctions pures sont infiniment plus testables. Une fonction pure ne dépend que de ses arguments — pas d'état global, pas de fichiers, pas de réseau. Pour la tester, il suffit d'appeler `f(entrée)` et de vérifier la sortie. Pas de `setUp`, pas de mocks, pas d'état à nettoyer. Quand j'ai commencé à écrire des fonctions pures systématiquement, mes tests sont devenus trois fois plus courts et dix fois plus lisibles.

2. `@lru_cache` peut transformer un programme. La suite de Fibonacci récursive calcule `fib(35)` en plusieurs secondes. Avec `@lru_cache`, c'est instantané. Cette mémoïsation automatique — mettre en cache les résultats déjà calculés — est l'une des optimisations les plus simples et les plus puissantes disponibles. Elle ne fonctionne que sur des fonctions pures, ce qui est une motivation supplémentaire pour les écrire.

3. La composition de fonctions est de l'algèbre appliquée. `pipeline(nettoyer, minuscules, tokeniser, compter)` n'est pas juste du code élégant — c'est la notation mathématique `compter tokeniser minuscules nettoyer` transposée en Python. Chaque fonction est une transformation pure, composable à l'infini. Ce paradigme change la façon de concevoir les problèmes : au lieu de "comment faire X", on demande "quelle séquence de transformations mène de l'entrée à la sortie".

Résumé du Chapitre 30

Ce que j'ai appris dans ce chapitre :

- **Fonctions pures** — même entrée, même sortie, zéro effet de bord.
- **Immutabilité** — ne jamais modifier les données d'entrée.
- `functools.partial` — application partielle, spécialiser une fonction.
- `@lru_cache` / `@cache` — mémoïsation automatique.
- `functools.reduce` — plier une séquence en une valeur.
- `operator.attrgetter`, `itemgetter`, `methodcaller`.
- **Composition** : `pipeline(f, g, h)` — chaîner des transformations.
- **Currying** — transformer `f(a,b,c)` en `f(a)(b)(c)`.
- **Maybe** — encapsuler les valeurs optionnelles sans `if None`.

*"La programmation fonctionnelle ne demande pas d'abandonner les objets ou les boucles. Elle demande de penser en **transformations** — et ça change tout."*